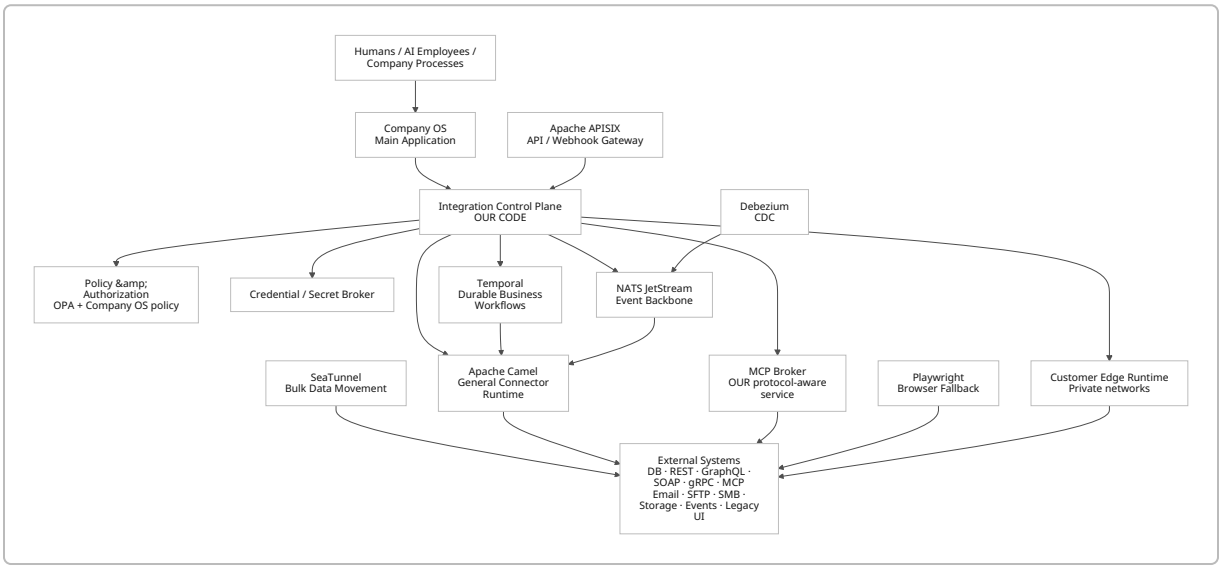
ChatGPT

Company OS Universal Integration Control Plane and Execution Architecture

Executive summary

The correct architecture for the Company OS is **not one large integration product** and not a direct collection of point-to-point connectors.

It should be a **Company OS-owned Integration Control Plane** above several specialized execution engines:



My recommendation after reviewing the current 2026 projects and releases is:

Responsibility	Recommended foundation	Decision
Integration product model	**Our code**	Must own
Connector/action/trigger contract	**Our code**	Must own
Tenant/company/portal policy	**Our code + OPA**	Must own
API ingress and webhook gateway	**Apache APISIX**	Reuse
General integration/protocol runtime	**Apache Camel**	Reuse
Durable multi-step workflows	**Temporal**	Reuse

Responsibility	Recommended foundation	Decision
Event backbone	**NATS + JetStream**	Reuse
Agent-facing external integrations	**Native MCP broker**	Build on MCP SDK/spec
Database change capture	**Debezium**	Reuse
Bulk/large-volume data synchronization	**SeaTunnel**	Add when needed
Browser-only legacy systems	**Playwright**	Last-resort worker
Visual integration designer	**Kaoto code/concepts**	Reuse selectively
Ready-made SaaS connector logic	**Activepieces Pieces**	Harvest/adapt selectively
Policy engine	**Open Policy Agent**	Reuse
SBOM generation	**Syft**	Reuse
Vulnerability scanning	**Grype or equivalent**	Reuse
Artifact signing	**Cosign**	Reuse

This design deliberately separates **what the Company OS believes an integration means** from **how a specific system happens to implement it**.

An AI employee should ask for:

```
crm.contact.search
crm.contact.update
invoice.read
invoice.create
document.search
email.send
calendar.event.create
project.create
```

It should not normally need to know that Company A uses REST, Company B uses PostgreSQL, Company C exposes MCP, and Company D can only be automated through a browser.

That abstraction is the critical product asset.

Main conclusions

Apache Camel should be the broad connector/protocol engine. Camel's current 4.18 LTS catalog includes JDBC/SQL, REST/OpenAPI, GraphQL, gRPC, Kafka, web services, SFTP and many other integration components; its REST OpenAPI component can consume OpenAPI 3.x descriptions. [1] The current supported LTS is 4.18.x; 4.18.3 was released in July 2026, while 4.21.0 is the newer feature release. The project also publishes CycloneDX SBOMs for modern releases. [2]

APISIX should be the north-south API/webhook gateway, not the integration engine. Version 3.17.0 was released June 15, 2026 and added stricter authentication/TLS behavior, new access controls, GraphQL traffic controls, data-protection features and other gateway improvements. It is Apache-2.0 licensed. [3]

Temporal should own long-lived orchestration. Temporal Server 1.31.2 is current as of August 10, 2026, is MIT licensed, and exists specifically to execute durable workflows that survive failures and retry failed activities. [4]

NATS JetStream should be the default internal event backbone, but we should design the business layer around **at-least-once delivery plus idempotency**, not advertise magical exactly-once business effects. JetStream provides persistence, replay, replicated streams and at-least-once consumption, and it provides message deduplication/double-ack mechanisms for stronger delivery semantics. Application-level idempotency is still necessary. [5]

MCP should be a first-class integration type but should not replace ordinary APIs. The July 28, 2026 MCP specification is the current release and moves the protocol toward a stateless HTTP-friendly architecture. MCP tools are explicitly designed to let models invoke external systems, and the specification requires serious consent and authorization controls. [6]

Debezium should handle database CDC rather than us inventing CDC. Debezium 3.6.1.Final was released August 4, 2026 and includes integrity/security fixes. The stable release supports major database families and can also be embedded through the Debezium Engine API. [7]

SeaTunnel should remain an optional specialist engine. Version 2.3.13 is current in the retrieved release stream and is designed for distributed, high-volume data integration. Its JDBC sink supports both batch and streaming execution and can use XA for stronger write guarantees where the database supports it. [8]

Playwright belongs at the bottom of the preference ladder. Playwright 1.62.1 is current as of this research, Apache-2.0 licensed, and supports Chromium, Firefox and WebKit. It is excellent automation infrastructure, but browser automation is inherently more fragile than a stable programmatic interface. [9]

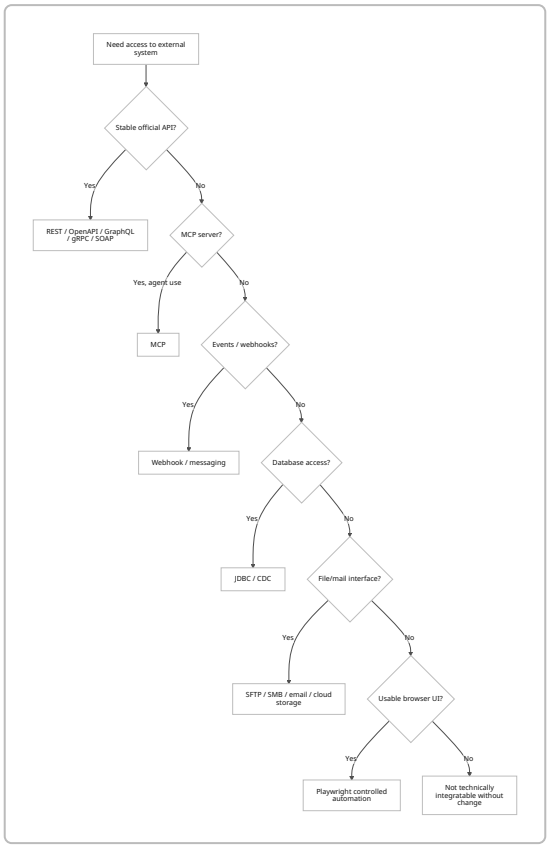
The most important deployment capability that should be added to the previous design is a **Customer Edge Integration Runtime**. Many of the businesses we want to serve will have databases, SMB shares, SFTP servers, old ERP systems and internal APIs that cannot safely be exposed to the Internet. The Company OS should therefore support a small customer-hosted runtime that makes **outbound authenticated connections to our control plane** and executes approved integrations inside the customer's network.

That dramatically expands what “integrate with anything” means without requiring customers to open inbound database ports to us.

Integration coverage and priorities

The platform should not treat all integration mechanisms as equal. There should be a preference order based on stability, security and maintainability.

Recommended integration preference



This ordering is not absolute. For example, CDC may be preferable to a poorly designed polling API, and MCP may be the official interface supplied by a modern AI-oriented product.

Priority tiers

Priority	Integration family	Typical purpose	Primary runtime
P0	REST / OpenAPI	Most SaaS and modern business systems	Camel
P0	Webhooks	Immediate incoming events	APISIX + Control Plane
P0	JDBC / SQL	Legacy/on-prem databases, analytics, controlled writes	Camel
P0	MCP	Agent-facing tools/resources	Native MCP Broker
P0	Email	Mailboxes as business interfaces	Camel
P0	Files / SFTP / SMB	Legacy exchange, imports/exports	Camel
P0	Cloud object/file storage	Documents, exports, archives	Camel/native connector

Priority	Integration family	Typical purpose	Primary runtime
P1	GraphQL	Schema-driven SaaS APIs	Camel/native HTTP
P1	gRPC	Internal services and modern infrastructure	Camel/APISIX
P1	SOAP / XML web services	ERP, financial, government and legacy enterprise	Camel
P1	Messaging	NATS, Kafka, AMQP/JMS, MQTT	Camel + native bus
P1	CDC	Near-real-time DB change streams	Debezium
P2	Bulk/ETL/ELT	Millions of rows / historical migrations	SeaTunnel
P2	Streaming data pipelines	Continuous high-volume movement	SeaTunnel / Kafka / NATS depending workload
P3	Browser automation	Systems with no usable interface	Playwright

Camel 4.18 LTS directly covers a large fraction of this table. Its component catalog includes GraphQL, gRPC, JDBC/SQL, Spring Web Services, REST/OpenAPI, Kafka and numerous SaaS/cloud integrations. Its broader catalog also lists Google Drive, Google Mail, Google Sheets, Google Storage and similar services. [10]

Its GraphQL implementation is currently a client supporting queries and mutations but not GraphQL subscriptions, so GraphQL subscriptions should be handled separately if a vendor requires them. [11]

Camel's REST OpenAPI support handles OpenAPI 3.x, not old Swagger 2.0, so the Connector SDK should contain a normalization/conversion step for legacy Swagger descriptions. [12]

What “integrate with anything” honestly means

There is no software platform that can literally integrate with every external system.

An external system needs at least one usable surface:

```
API
OR
MCP
OR
event/webhook
OR
database protocol
OR
message protocol
OR
```

```
mail
OR
file exchange
OR
browser/interface that can be automated
```

If a system is physically unreachable, exposes no interface, and cannot be automated through its UI, integration is impossible until something changes at that system.

Our goal should therefore be:

> **Support every major class of interface, with a controlled fallback ladder.**

Direct database integration

For relational systems, JDBC should be our generic protocol.

Camel's JDBC and SQL components let routes execute database operations through a supplied datasource. [13]

A database connection should contain:

```
DatabaseConnection
├── engine / driver
├── host
├── port
├── database
├── TLS policy
├── credential_ref
├── allowed schemas
├── allowed tables
├── allowed operations
├── query timeout
├── concurrency limit
├── connection pool policy
└── execution location
```

Execution location is important:

```
Cloud reachable database
    → Company OS connector worker

Private database
    → Customer Edge Runtime
```

```
High-security database
    → Dedicated tenant worker
```

For AI-facing access, arbitrary SQL should **not** be the default.

Instead:

```
AI Employee
    ↓
finance.invoice.search
    ↓
Capability Service
    ↓
Approved parameterized query
    ↓
Database
```

This avoids giving a model unrestricted SQL authority.

REST and OpenAPI

REST/OpenAPI should be the easiest connector type to create.

An import wizard should accept:

```
OpenAPI URL
OpenAPI file
API base URL
Authentication configuration
```

Then generate draft:

- Operations.
- Input/output schemas.
- Capabilities.
- Authentication requirements.
- Retry rules.
- Rate-limit fields.
- Test requests.

Camel can construct REST producers from OpenAPI 3.x definitions. [12] APISIX 3.17 also provides OpenAPI request validation through its OAS validator functionality, which is useful at the ingress boundary. [14]

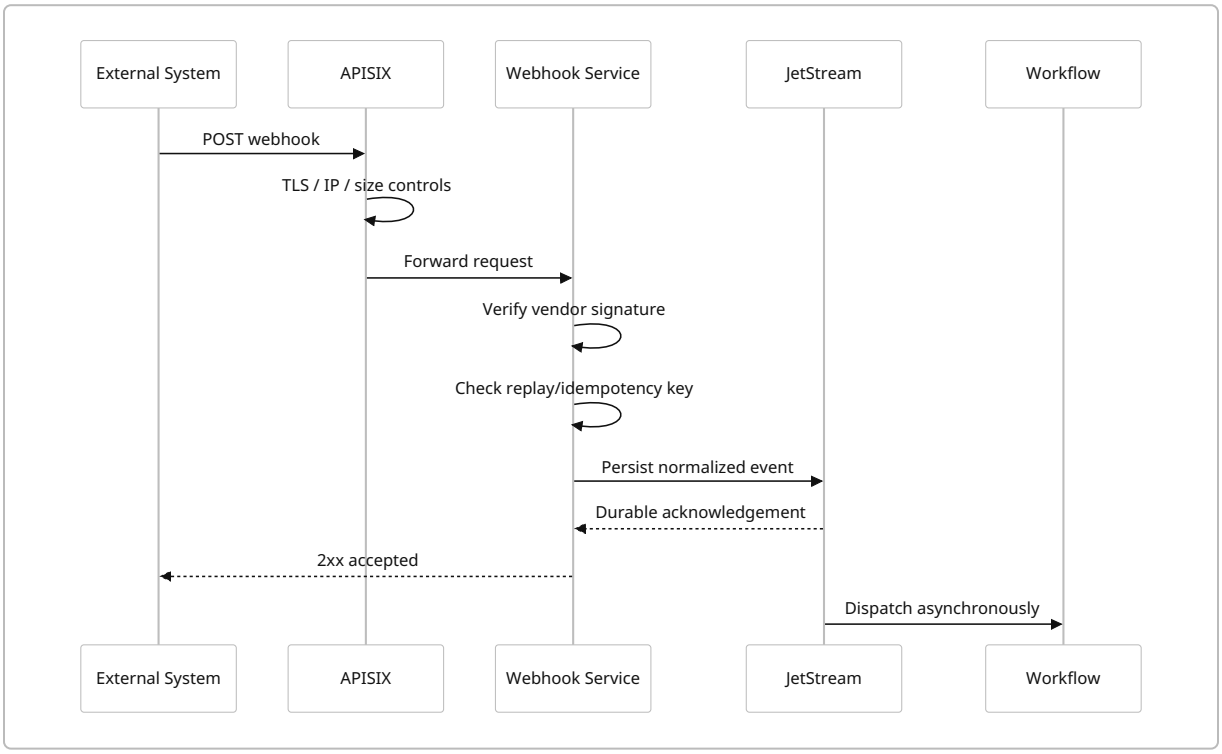
Generated operations must still be reviewed. An OpenAPI document generally does not tell us enough about:

- Business side effects.
- Idempotency.
- Approval risk.
- Vendor quotas.
- Semantic meaning.
- Data sensitivity.

Those belong in our connector manifest.

Webhooks

Inbound webhooks should terminate at APISIX and our Webhook Ingress Service:



Where vendors support signed webhooks, the connector should supply signature verification.

For example, Frappe's current webhook mechanism can add an `X-Frappe-Webhook-Signature` containing an HMAC-SHA256 signature when a webhook secret is configured. [15]

Email and files

Mail/file interfaces deserve first-class status rather than being treated as old-fashioned exceptions.

A business integration might be:

```
New message arrives in invoices@company.com
        ↓
Validate sender
        ↓
Extract PDF
        ↓
Run document extraction
        ↓
Match supplier
        ↓
Create review task
        ↓
Post approved invoice
```

Similarly:

```
Supplier writes:
/outbound/orders_2026-08-10.csv
        ↓
SFTP watcher
        ↓
Validate checksum and schema
        ↓
Import
        ↓
Archive
        ↓
Emit OrderImportCompleted
```

Camel supplies SFTP support and related file-transfer components. [16]

Public APIs for our own platform

A universal integration platform must itself be easy to integrate **into**.

Company OS should expose:

- REST/OpenAPI.
- Webhooks.
- MCP server.
- Event subscriptions.
- Bulk export/import.
- Eventually gRPC for internal/high-volume clients.

This creates two directions:

```
External software
      ↕
Integration Fabric
      ↕
Company OS
```

not merely:

```
Company OS
      ↓
External software
```

Recommended open-source stack and deployment architecture

The following versions/statuses are current to the research date, **August 10, 2026**.

Project	Current/recommended status	License	Role in our architecture	Source
Apache Camel	**4.18.3 LTS** recommended baseline; 4.21.0 current newer line	Apache-2.0	General connector/ protocol runtime	Downloads [2]
Apache APISIX	**3.17.0**, June 15, 2026	Apache-2.0	API/webhook ingress, routing, auth, throttling	3.17 release [3]
Debezium	**3.6.1.Final**, August 4, 2026	Apache-2.0	CDC	3.6.1 release [17]
Temporal Server	**1.31.2**, July 8, 2026	MIT	Durable orchestration	Repository [4]
NATS Server	**2.14.3**, June 29, 2026	Apache-2.0	Internal event backbone / JetStream	Releases [18]
Apache SeaTunnel	**2.3.13**, March 2026	Apache-2.0	Bulk/stream data movement	Releases [19]
Playwright	**1.62.1**	Apache-2.0	Browser automation fallback	Repository [9]
Kaoto	**2.11**, June 17, 2026	Apache-2.0	Visual Camel design / mapping inspiration or reuse	Repository [20]

Project	Current/recommended status	License	Role in our architecture	Source
Activepieces	**0.87.0**, August 6, 2026	MIT Community + commercial EE sections	Reusable TypeScript connector logic	Releases [21]
MCP specification	**2026-07-28**	MIT specification repository	Agent integration standard	Specification [22]
Open Policy Agent	Active project	Apache-2.0	Fine-grained policy evaluation	Repository [23]

Apache Camel

Recommended role: generic execution adapter.

Use Camel for:

- HTTP.
- REST/OpenAPI.
- JDBC/SQL.
- SOAP.
- GraphQL.
- gRPC.
- SFTP/FTP.
- Mail.
- Kafka/JMS.
- Cloud storage.
- SaaS connectors.
- Enterprise protocols.

Camel should **not own**:

- Tenants.
- Company roles.
- Human/AI identities.
- Connections.
- Secrets.
- Business workflow definitions.
- Company-level permissions.

It should receive a narrow execution request:

```
{
  "runId": "run_123",
  "tenantId": "tenant_a",
  "connectionHandle": "execconn_x",
```

```
  "connector": "erpnext",
  "connectorVersion": "2.1.0",
  "capability": "finance.invoice.read",
  "input": {
    "invoiceId": "INV-1008"
  }
}
```

and return a normalized result.

Strengths: extremely broad protocol coverage; mature routing patterns; support for circuit breakers and saga patterns; several deployment forms; OpenAPI tooling. Camel provides a Circuit Breaker EIP and Saga EIP in its current documentation. [24]

Weaknesses: Java ecosystem; significant surface area; easy to create overly complex routes; it is an integration framework rather than our desired product/control plane.

Version policy: use the newest patched LTS rather than blindly using the initial LTS release. For example, Camel's advisory for CVE-2026-46590 affected earlier 4.18 releases and is fixed in 4.18.3, illustrating why patch-level upgrades matter. [25]

Apache APISIX

Recommended role: public protocol boundary.

Use APISIX for:

- Public Company OS API.
- Incoming webhooks.
- External MCP-over-HTTP front door.
- Rate limits.
- Authentication adapters.
- TLS.
- Request-size rules.
- IP rules.
- Canary routing.
- OpenAPI validation.
- Gateway metrics.

APISIX's OIDC plugin can authenticate through OpenID Connect providers, and its OAS validator can validate requests against OpenAPI specifications. [26]

Do not turn APISIX into the entire authorization system. Gateway authorization is only the first layer.

Temporal

Recommended role: orchestration when business state must survive time and failure.

Example:

```
CustomerContractSigned
    ↓
Create CRM customer
    ↓
Create ERP customer
    ↓
Wait for Finance approval
    ↓
Create project
    ↓
Create document folder
    ↓
Assign onboarding AI
    ↓
Wait up to 72 hours for missing documents
    ↓
Escalate if necessary
```

The workflow might exist for days. Temporal is designed for durable workflows and automatic retries after intermittent failures. [4]

Temporal should not become the generic event bus or connector registry.

Namespaces can provide one technical level of logical isolation in a self-hosted Temporal deployment, but Company OS authorization must still be enforced above them. [27]

NATS and JetStream

Recommended role: internal event distribution.

Examples:

```
customer.created
invoice.updated
task.completed
integration.connection.failed
knowledge.updated
ai.action.completed
```

Use plain Core NATS only for ephemeral notifications where loss is acceptable.

Use JetStream whenever an event must be durable/replayable.

JetStream supplies persistent streams and consumers and is built for durable message processing. [28]

For critical flows, use three replicas at minimum as an architectural baseline, and test filesystem durability behavior explicitly. JetStream documentation discusses the tradeoff between replication and disk synchronization; stronger disk synchronization can be selected at a performance cost. [28]

Debezium

Recommended role: change capture from databases we do not control.

Example:



Debezium's stable connector set covers PostgreSQL, MySQL, MariaDB, SQL Server, Oracle, Db2, MongoDB and other supported stores. [29]

SeaTunnel

Recommended role: exceptional volume.

Do not route a 500-million-row migration through ordinary interactive Camel actions.

Use SeaTunnel for:

- Initial migration.
- Data warehouse loads.
- Large table copies.
- High-volume continuous synchronization.
- Large file pipelines.

SeaTunnel 2.3.13 supports distributed data-integration workloads and JDBC batch/stream sinks. [30]

Playwright

Recommended role: explicit fallback.

It should run in a dedicated browser-worker tier, not inside an API server or AI-agent runtime.

Playwright currently supports Chromium, Firefox and WebKit through one automation framework, and the current project is Apache-2.0. [9]

Kaoto

Kaoto is a useful source of:

- Camel flow visualization.
- Component property editors.
- Data mapping UI.
- Testing UI.
- Kamelet/component discovery.

The project describes access to hundreds of Camel components/Kamelets and visual data mapping, and is Apache-2.0 licensed. [31]

I would reuse or learn from Kaoto rather than expose Kaoto itself as the customer integration product.

Activepieces

Activepieces should be treated as a **connector source ecosystem**, not our architectural kernel.

Its Pieces system is TypeScript-based and the project reports more than 280 pieces; the Community Edition is MIT while enterprise areas are commercially licensed. [32] Version 0.87.0 was released August 6, 2026 and includes ongoing connector, OAuth, MCP, sandbox and workflow work. [33]

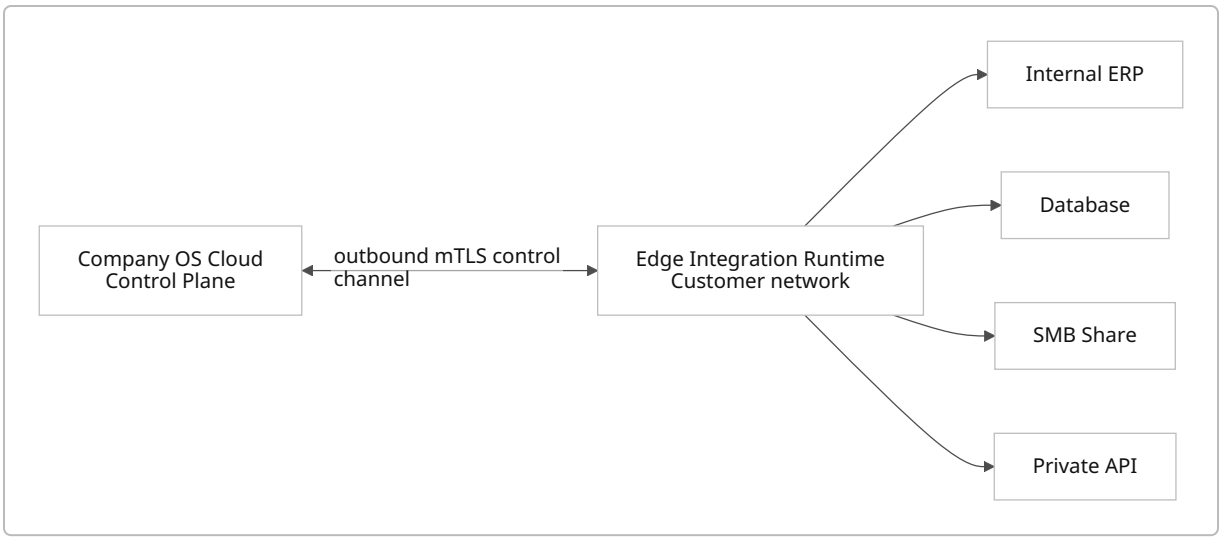
The rule should be:

```
Activepieces piece
    ↓
license/dependency review
    ↓
code/API recertification
    ↓
our compatibility wrapper
    ↓
our Connector Contract
```

Do not make our `Connection`, `Tenant`, `Policy`, `Run` or `Flow` objects depend on Activepieces internals.

Private-network Edge Runtime

This should be an important proprietary part of the architecture.



The customer should **not** normally need to expose:

```
5432 PostgreSQL
3306 MySQL
445 SMB
22 SFTP
internal ERP API
```

to the public Internet.

The Edge Runtime:

- Makes an outbound connection.
- Receives signed job/capability definitions.
- Gets only temporary credential handles.
- Runs selected Camel/native connectors.
- Sends normalized results/events back.
- Has no ability to browse unrelated tenant workloads.
- Can optionally execute Debezium locally.
- Can be customer-managed for high-security installations.

Support three deployment classes:

Class	Deployment
Standard SaaS	Shared control plane + multi-tenant isolated workers
Regulated/dedicated	Shared control plane + dedicated execution namespace/node pool
Private/on-prem	Edge runtime inside customer's network

Control plane, connector contract, security, tenancy, secrets, and MCP governance

The **Integration Control Plane is the part we should build ourselves**.

None of Camel, Temporal, APISIX, Activepieces, Debezium or SeaTunnel knows enough about our:

- Company.
- Portal.
- Departments.
- Human employees.
- AI employees.
- Delegation rules.
- Approval rules.
- Company knowledge.
- AI authority.
- Business context.

Core control-plane objects

The minimum durable model should contain:

Object	Purpose
`Connector`	Stable logical connector identity
`ConnectorVersion`	Immutable implementation version
`Connection`	Tenant-owned configuration for one external system
`ConnectionCredentialRef`	Reference to encrypted credentials
`Capability`	Normalized action/query/event
`Trigger`	Source of events/work
`WorkflowDefinition`	Business orchestration definition
`WorkflowVersion`	Immutable workflow revision
`Mapping`	Transformation definition
`MappingVersion`	Immutable mapping revision
`ExecutionPolicy`	Limits, approvals and risk rules
`WebhookEndpoint`	Managed incoming endpoint
`Run`	One workflow/integration execution
`StepRun`	One execution step

Object	Purpose
`Checkpoint`	Poll/CDC synchronization position
`ExternalRecordLink`	Maps IDs across systems
`MCPServerConnection`	External MCP endpoint
`MCPToolSnapshot`	Approved tool catalog/version
`Artifact`	Connector image/package and digest
`EdgeRuntime`	Registered private execution node

A `Connection` should never mean merely "a set of credentials."

It should be:

```
Connection
├── tenant
├── owner/scope
├── connector
├── connector version policy
├── environment
├── endpoint
├── auth method
├── credential reference
├── enabled capabilities
├── rate limits
├── execution location
├── network restrictions
├── health
├── data classification
└── policy
```

Capability abstraction

The most strategically important model is `Capability`.

Vendor operations can map to a semantic capability:

```
Twenty API
POST /rest/companies
          │
          ▼
crm.company.create
```

```
ERPNext API
POST /api/resource/Customer
          │
          ▼
customer.create


Custom PostgreSQL procedure
SELECT create_customer(...)
          │
          ▼
customer.create
```

Not every capability should be universal. Vendor-specific capabilities remain valid:

```
salesforce.lead.convert
notion.database.query
sap.purchase_order.release
```

The goal is semantic reuse where it provides value, not an impossible universal schema.

Proposed connector manifest

```yaml
apiVersion: companyos.io/integration/v1
kind: Connector

metadata:
  id: twenty
  name: Twenty CRM
  version: 1.4.0
  publisher: companyos
  license: proprietary-wrapper
  implementationDigest: "sha256:..."
  maturity: certified

runtime:
  type: camel
  image: registry.companyos.io/connectors/twenty@sha256:...
  minimumRuntimeVersion: "1.3"
  timeoutSeconds: 30

network:
  egress:
```

```yaml
    - scheme: https
      hosts:
        - "*.twenty.com"
  privateNetworkAllowed: true

connections:
  - id: api-key
    type: bearer
    fields:
      - name: baseUrl
        secret: false
        required: true
      - name: apiKey
        secret: true
        required: true

capabilities:
  - id: crm.company.read
    kind: query
    vendorOperation: company.get
    inputSchema: schemas/company-get.input.json
    outputSchema: schemas/company.json

    sideEffect: none
    retry:
      safe: true
      maxAttempts: 4

    idempotency:
      mode: not-required

    authorization:
      defaultRisk: low

  - id: crm.company.create
    kind: action
    vendorOperation: company.create
    inputSchema: schemas/company-create.input.json
    outputSchema: schemas/company.json

    sideEffect: reversible_write

    idempotency:
      mode: idempotency-key
      scope: connection

    retry:
      safe: true
```

```
      maxAttempts: 4
      retryOn:
        - timeout
        - 429
        - 502
        - 503

    authorization:
      defaultRisk: medium
      aiAllowed: policy

rateLimits:
  strategy: token-bucket
  scope: connection
  providerHeaders:
    retryAfter: Retry-After

health:
  capability: crm.company.read
  intervalSeconds: 300
```

Side-effect classification

Every executable capability must have a declared side effect.

Class	Meaning	Example
`none`	Read only	Search contacts
`local_write`	Only our state changes	Save mapping
`reversible_write`	External change can normally be reverted	Update CRM field
`external_communication`	Sends something outside company	Send email
`financial`	Moves/commits financial value	Issue refund
`destructive`	Deletes or irreversibly changes	Delete contract
`security_sensitive`	Access/security change	Create API key

These classifications become inputs to policy.

Example:

```
IF actor.type = AI
AND capability.sideEffect = financial
```

```
AND amount > 500 EUR
THEN require FinanceManager approval
```

Idempotency contract

Every write capability must declare one of:

```
natural-key
provider-idempotency-key
upsert
deduplicated-by-read-before-write
transactional
unsafe
```

`unsafe` means automatic retry is disabled after an ambiguous outcome unless an explicit compensation/reconciliation routine exists.

Example:

```
idempotency:
  mode: provider-idempotency-key
  header: Idempotency-Key
  platformKey:
    format: "{tenantId}:{runId}:{stepId}"
```

Rate limiting

Rate-limit metadata belongs in the connector definition and can be overridden by a connection.

```
rateLimits:
  - scope: connection
    requests: 100
    period: PT1M

  - capability: crm.search
    concurrency: 5
```

The runtime should support:

- Token bucket.
- Fixed window when vendor semantics require it.
- Max concurrency.
- Per-connection limit.

- Per-tenant limit.
- Global vendor limit.

`429` responses should honor `Retry-After` when supplied.

Twenty's current API documentation, for example, publishes a 100-requests-per-minute limit and up to 60 records per batch, illustrating why these rules should live in connector metadata rather than generic global defaults. [34]

Human, AI and service principals

Every execution needs an actor context.

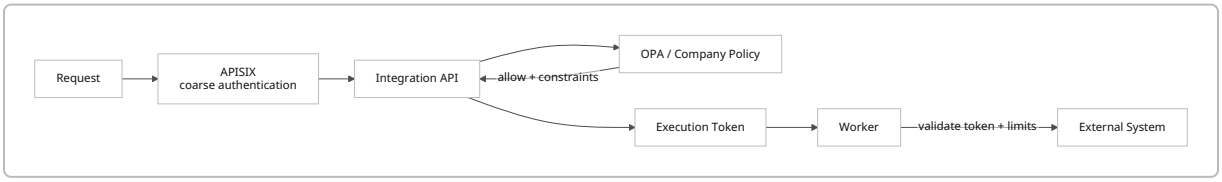
```
{
  "principal": {
    "id": "agent_finance_001",
    "type": "ai_employee"
  },
  "tenantId": "tenant_492",
  "portalId": "portal_finance",
  "scopeId": "project_882",
  "delegation": {
    "grantedBy": "user_19",
    "expiresAt": "2026-08-10T18:00:00Z"
  },
  "correlationId": "corr_..."
}
```

The worker must **not trust** `tenantId` **merely because it appears in JSON**.

A signed short-lived execution token should carry the authoritative claims.

Policy enforcement

Use several enforcement points:



Open Policy Agent is Apache-2.0 and designed as a general-purpose context-aware policy engine, making it suitable as a policy-decision component behind our own policy model. [23]

Policy inputs can include:

```
actor
tenant
portal
department
role
AI/human/service
delegation
capability
side-effect classification
resource
customer
project
data classification
amount
time
location
connection
```

Policy outputs should be more expressive than Boolean allow/deny:

```
{
  "decision": "allow",
  "constraints": {
    "maxRecords": 50,
    "readOnly": true,
    "requireApprovalAbove": 1000,
    "maskFields": ["salary", "nationalId"]
  }
}
```

Secrets design

The Control Plane database should contain only references:

```
Connection
credential_ref = cred_x7f9
```

Never:

```
Connection
password = "..."
refresh_token = "..."
```

The secret subsystem should support a provider-neutral interface:

```
interface SecretProvider {
  putSecret(...): Promise<SecretRef>;
  getForExecution(...): Promise<ExecutionCredential>;
  rotate(...): Promise<void>;
  revoke(...): Promise<void>;
}
```

Recommended rules:

- Envelope encryption with KMS/HSM-backed master keys in production.
- Separate encryption context by tenant.
- Encrypt OAuth refresh tokens.
- Never return long-lived credentials to the browser.
- Never include credentials in AI prompts.
- Never include credentials in event payloads.
- Never include credentials in logs/traces.
- Inject credentials only into the worker that requires them.
- Prefer short-lived credentials/tokens.
- Revoke execution grants immediately when a connection is disabled.

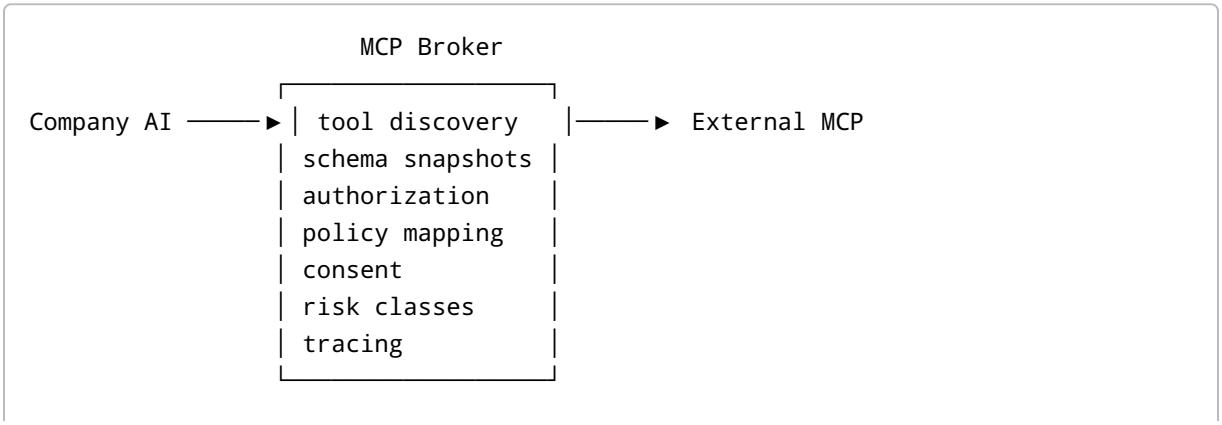
For a database, credentials should ideally belong to a dedicated user with:

```
read-only
or
specific schema/table/procedure grants
```

rather than a database administrator.

MCP architecture

MCP deserves its own protocol-aware service:



```
External MCP Client
        │
        ▼
   APISIX
        │
        ▼
Company OS MCP Server
        │
        ▼
Capability Layer
```

This gives us **both directions**:

1. Company OS consumes MCP servers.
2. Company OS itself exposes governed capabilities as MCP tools/resources.

The current MCP specification is version `2026-07-28`, and the specification repository is MIT licensed. [35]

The current specification defines MCP tools as model-controlled operations that may interact with databases, APIs and computations. [36]

That makes an important distinction:

> MCP is an excellent agent interface. It is not automatically the safest interface for deterministic business transactions.

If an ERP provides both:

```
POST /invoices
```

and:

```
MCP tool: create_invoice
```

a deterministic workflow should generally call the direct normalized capability.

An AI employee can use the MCP tool when model-directed interaction is genuinely useful.

MCP security requirements

Current MCP authorization requires OAuth 2.1 for authorization servers and Protected Resource Metadata for discovery. [37]

Our broker should enforce:

```
Server registration
    ↓
Protocol discovery
    ↓
Authentication validation
    ↓
List tools/resources
    ↓
Snapshot schemas
    ↓
Risk classification
    ↓
Admin/policy approval
    ↓
Expose approved subset
```

The MCP specification explicitly says tool descriptions/annotations should be considered untrusted unless they come from a trusted server and requires hosts to build consent and access controls around tool invocation. [38]

Therefore:

Never trust an MCP server's declaration that an operation is harmless.

Our catalog determines:

```
read
write
external communication
financial
destructive
security-sensitive
```

When a tool catalog changes:

```
Existing approved tool:
notion.search

New tools discovered:
notion.delete_database
notion.share_workspace
```

the new capabilities should remain unavailable until policy/review permits them.

MCP also explicitly forbids unsafe credential passthrough patterns for third-party authorization. Current guidance says third-party credentials must not transit through the MCP client and that the MCP server should manage its separate third-party credentials. [39]

For remote MCP:

- Prefer the current HTTP transport.
- Put APISIX in front for transport-level security.
- Put our MCP Broker behind APISIX for protocol policy.

For local `stdio` MCP:

- Run in isolated connector/edge workers.
- Never launch arbitrary community MCP packages in the central API process.
- Restrict filesystem.
- Restrict environment variables.
- Restrict egress.
- Pin exact package/artifact versions.

MCP's current specification also uses JSON Schema and recommends JSON Schema 2020-12 when no different dialect is declared, which aligns well with our connector-schema direction. [40]

Execution models, reliability, CDC, transformations, and browser fallback

There should not be one execution mode.

Runtime model matrix

Model	Example	Engine	Response model
Synchronous	Read customer	Camel/native worker	Immediate
Async command	Generate large report	Temporal/worker	`202 + runId`
Durable workflow	Customer onboarding	Temporal	Long-running
Webhook	Invoice updated	APISIX → JetStream	Fast acceptance
Polling	Check new records every 5 min	Temporal/Camel scheduler	Event output
Event subscription	Customer changed	JetStream	Async
CDC	SQL row changed	Debezium	Continuous

Model	Example	Engine	Response model
Batch	Import 50m rows	SeaTunnel	Job
Streaming	Continuous high-volume feed	SeaTunnel/Kafka/NATS	Continuous
MCP	AI invokes Notion tool	MCP Broker	RPC/tool invocation
Browser	Old portal form	Playwright worker	Usually async

Synchronous actions

Use only when:

- Work is fast.
- External provider response is reasonably bounded.
- No human approval is involved.
- One immediate result is useful.

Recommended API shape:

```
POST /v1/capabilities/crm.company.read:execute
```

Response:

```
{
  "runId": "run_123",
  "status": "succeeded",
  "result": {
    "id": "company_77",
    "name": "Example AB"
  }
}
```

Asynchronous actions

If execution can take substantial time:

```
POST /v1/capabilities/report.generate:execute
```

returns:

```
{
  "runId": "run_882",
  "status": "queued"
}
```

The client then uses:

```
GET /v1/runs/run_882
```

or receives an event.

Reliability baseline

The safest general assumption is:

> **Messages and work may be delivered more than once.**

This is especially important for CDC. Debezium's SQL Server documentation explicitly notes scenarios where source records can be emitted more than once because offsets are persisted periodically. [41]

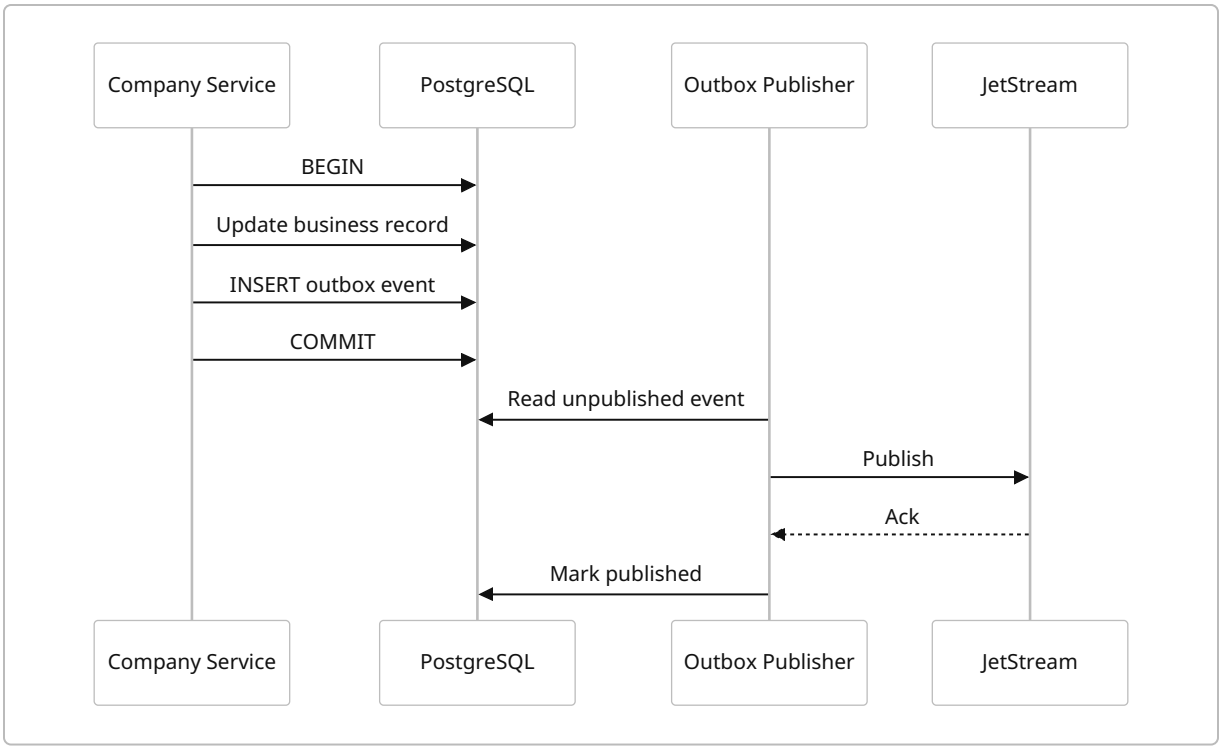
Therefore:

```
at-least-once delivery
+
idempotent consumers
+
deduplicated business writes
=
our baseline
```

Do not market broker-level “exactly once” as equivalent to exactly one business effect.

Transactional outbox

Every important Company OS database transaction that emits an event should use an outbox.



This avoids the classic failure:

```
DB commit succeeds
↓
process crashes
↓
event never published
```

Consumer inbox/deduplication

Consumers should maintain:

```
processed_message
├── tenant_id
├── consumer
├── message_id
├── idempotency_key
├── first_seen_at
└── result_reference
```

with a uniqueness constraint.

Pseudo-code:

```
await db.transaction(async (tx) => {
  const inserted = await tx.inbox.tryInsert(message.id);

  if (!inserted) {
    return; // duplicate
  }

  await performBusinessEffect(tx, message);
});
```

Idempotency keys

Recommended composition:

```
tenant_id
+
connection_id
+
capability_id
+
business operation key
```

or workflow-scoped:

```
tenant_id:workflow_run_id:step_id
```

Do not generate a fresh idempotency key on each retry.

Failure taxonomy

Every worker should normalize errors:

Class	Typical cases	Retry
`transient_network`	Timeout, connection reset	Yes
`rate_limited`	HTTP 429	Yes, delayed
`upstream_unavailable`	502/503	Yes
`authentication_expired`	Refreshable OAuth	Refresh then retry
`authentication_revoked`	User revoked access	No; intervention
`permission_denied`	403	No

Class	Typical cases	Retry
`validation_error`	400/schema mismatch	No
`not_found`	Missing external object	Usually no
`conflict`	Version/write conflict	Policy dependent
`ambiguous_write`	Connection lost after submit	Reconcile before retry
`permanent`	Unsupported operation	No

DLQ design

Dead letters need a durable reference, not an opaque broker dump.

```
{
  "eventId": "evt_92",
  "runId": "run_88",
  "tenantId": "tenant_a",
  "connector": "erpnext",
  "connectionId": "conn_7",
  "errorClass": "validation_error",
  "attempts": 4,
  "payloadRef": "encrypted-object://...",
  "firstFailure": "...",
  "lastFailure": "...",
  "replayAllowed": true
}
```

The UI should support:

```
Inspect
Retry
Replay with new connector version
Edit mapping and replay
Discard with reason
Escalate
```

Circuit breakers

Maintain circuit state per:

```
vendor + connection
```

not only globally.

Company A's broken Salesforce credentials should not stop Company B.

Camel exposes a Circuit Breaker EIP through its current integration patterns. [42]

CDC deployment patterns

There are two sensible Debezium patterns.

Standard/high-scale pattern:

```
Database
    ↓
Debezium Kafka Connect
    ↓
Kafka
    ↓
Normalizer
    ↓
Company event model
```

Debezium is historically centered around Kafka Connect, and its documentation continues to describe this architecture. [43]

Smaller/edge deployment:

```
Database
    ↓
Edge CDC service
    ↓
Debezium Engine
    ↓
Normalizer
    ↓
JetStream / Company OS
```

Debezium exposes an embeddable Engine API, including asynchronous engine implementations. [44]

The second option avoids forcing Kafka into every small installation, but it transfers more responsibility to us.

Debezium connectors require persistent offset/state storage, and some connectors also maintain schema history that must survive restarts. [45]

Therefore the Edge Runtime must never treat CDC offsets as temporary container files.

CDC object model

```
CDCConnection
├── database_connection
├── connector_type
├── selected schemas
├── selected tables
├── snapshot mode
├── publication/replication settings
├── offset store
├── schema history store
├── event mapping
├── data classification
├── masking rules
└── lag thresholds
```

Raw CDC events should not automatically become public Company OS events.

Pipeline:

```
Debezium event
    ↓
Schema validation
    ↓
Table allowlist
    ↓
Sensitive field masking
    ↓
Source-specific normalization
    ↓
Business event mapping
    ↓
Company event
```

Example:

```
postgres.public.invoice UPDATE
```

becomes:

```
{
  "type": "finance.invoice.updated",
  "tenantId": "tenant_1",
  "source": {
    "system": "legacy_erp",
    "table": "invoice",
    "id": "9918"
  },
  "changes": {
    "status": {
      "before": "sent",
      "after": "paid"
    }
  }
}
```

CDC versus transactional outbox

For databases **we own**, prefer an application transaction/outbox.

For external systems **we cannot modify**, CDC is appropriate.

Do not use CDC as an excuse to ignore semantic application events.

A row changing:

```
status = 4
```

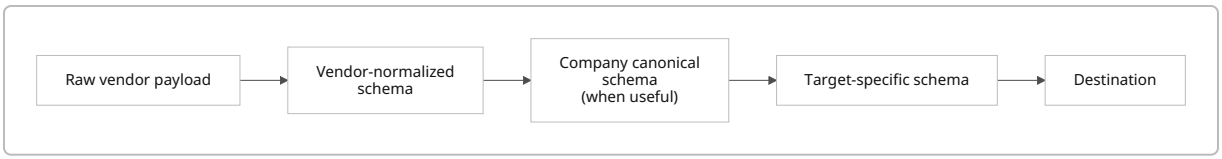
does not necessarily tell us whether that means:

```
invoice paid
invoice cancelled
invoice disputed
```

That requires a mapping.

Data transformation architecture

Use layered transformations:



Not every integration needs `C`.

For example:

```
Notion page
→ internal AI context
```

may remain vendor-specific.

But:

```
Twenty Company
→ Company OS Company
→ ERPNext Customer
```

benefits from a canonical business object.

Do not build one universal data model

Attempting to normalize every field in every business system will create an enormous abstraction that never quite fits.

Canonicalize high-value domains:

- Person.
- Company.
- Address.
- Product.
- Invoice.
- Project.
- Task.
- File/document.
- Employee.
- Event.

Allow extension fields:

```
{
  "id": "...",
```

```
  "name": "...",
  "extensions": {
    "twenty": {
      "customRiskScore": 7
    }
  }
}
```

Mapping definition

```
apiVersion: companyos.io/mapping/v1
kind: Mapping

metadata:
  id: twenty-company-to-canonical
  version: 3

source:
  schema: connector://twenty/company/v2

target:
  schema: canonical://crm/company/v1

rules:
  - from: id
    to: externalIds.twenty

  - from: name
    to: name

  - expression: |
      source.address?.country ?? null
    to: address.country

  - expression: |
      normalizeDomain(source.domainName)
    to: websiteDomain
```

The mapping runtime should provide:

- Field mapping.
- Constants.
- Conditional expressions.
- Array operations.
- Date/time normalization.
- Currency/decimal handling.

- Lookup tables.
- Reference resolution.
- Controlled reusable functions.

For complex transformations, allow signed code-based transforms, but treat them as connector artifacts with the same security process.

Kaoto's current visual DataMapper work is worth studying/reusing for the mapping editor rather than starting its interaction design from scratch. [20]

Mapping tests

Every mapping version should support fixtures:

```
tests:
  - name: normal-company
    input: fixtures/twenty-company.json
    expected: fixtures/company.expected.json

  - name: missing-address
    input: fixtures/twenty-company-no-address.json
    expected: fixtures/company-no-address.expected.json
```

Tests should include:

- Missing fields.
- `null`.
- Unicode.
- Timezones.
- DST boundaries.
- Decimal precision.
- Currency.
- Empty arrays.
- Very large strings/files.
- Unexpected vendor fields.
- Deleted fields.
- Schema-version upgrades.
- PII redaction.

Browser automation fallback

Browser automation should be marked internally as a less reliable connector class.

```
browserReliabilityClass: fallback
```

Every browser connector should declare:

```
browser:
  allowedHosts:
    - legacy.example.com

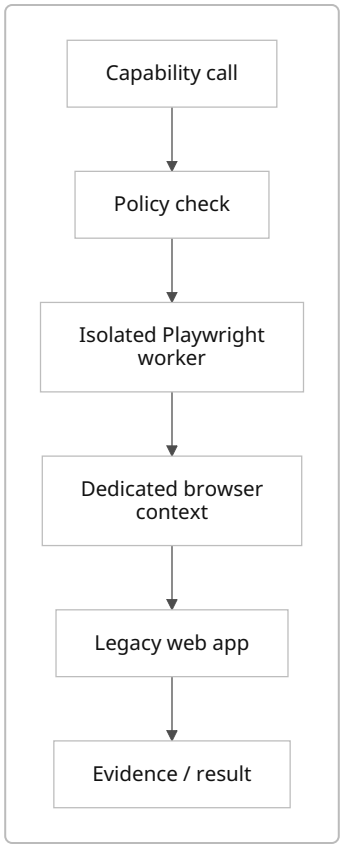
  session:
    persistent: true
    maxAge: PT8H

  downloads:
    allowed: true
    maxSizeMb: 50

  uploads:
    allowed: false

  evidence:
    screenshotOnWrite: true
    traceOnFailure: true
```

Execution:



Rules:

- Ephemeral worker wherever possible.
- Dedicated browser context per connection.
- No shared cookies across tenants.
- Domain allowlist.
- Block access to cloud metadata and internal infrastructure.
- No unrestricted filesystem.
- Credentials injected only into the worker.
- Prefer deterministic locators and scripts over unconstrained AI clicking.
- Require stronger review for write operations.
- Capture screenshots/traces of important actions.
- Run synthetic tests regularly to detect UI breakage.

Playwright's current project also includes browser/agent interoperability and MCP-related tooling, but we should use its browser automation APIs as the fundamental primitive rather than couple business-critical integration logic to its agent layer. 46

Observability, CI/CD, supply chain, SLAs, and upgrades

Integrations fail in more ways than ordinary application requests.

They depend on:

- Our software.
- Our network.
- Customer networks.
- Vendor networks.
- Vendor APIs.
- Credentials.
- OAuth consent.
- Vendor schema changes.
- Mapping versions.
- External quotas.
- Databases.
- Human approvals.

Observability therefore has to be part of the integration model.

Correlation model

Every operation should carry:

```
trace_id
correlation_id
run_id
```

```
step_run_id
tenant_id
portal_id
actor_id
workflow_id
connector_id
connector_version
connection_id
capability_id
idempotency_key
external_request_id
```

Example:

```
{
  "traceId": "7f...",
  "correlationId": "customer_onboarding_982",
  "runId": "run_221",
  "stepRunId": "step_7",
  "tenantId": "tenant_a",
  "actorId": "ai_sales_9",
  "connectorId": "twenty",
  "connectorVersion": "1.4.0",
  "connectionId": "conn_17",
  "capability": "crm.company.create",
  "idempotencyKey": "tenant_a:run_221:step_7"
}
```

Use OpenTelemetry for distributed tracing/log/metric correlation and standard W3C trace propagation between services. OpenTelemetry's context propagation is designed to correlate distributed telemetry, and W3C Trace Context provides the standard `traceparent` mechanism for passing trace identity. [47]

Do not put:

- Passwords.
- Access tokens.
- Customer PII.
- Full document contents.

into trace baggage.

Recommended telemetry

Category	Metrics
Gateway	Requests, 4xx, 5xx, latency, blocked requests

Category	Metrics
Connections	Health, auth expiry, token-refresh failures
Connector	Success %, p50/p95/p99, retries
Vendor	429 count, 5xx count, latency
Workflow	Running, completed, failed, waiting
Queue	Pending, age, redelivery, DLQ
CDC	Source lag, events/sec, last offset, schema errors
Batch	Rows/sec, bytes/sec, checkpoint
Browser	Login failures, selector failures, navigation errors
MCP	Tool discovery changes, call count, rejected tools
Policy	Allow, deny, require approval
Security	Invalid signatures, blocked egress, secret-access failures

Run trace UI

An engineer or customer administrator should see:

```
RUN #882
Customer onboarding

09:00:01  Trigger received                  OK
09:00:01  Signature verified                OK
09:00:01  Workflow started                  OK
09:00:02  Twenty: find company              OK  180 ms
09:00:02  ERPNext: create customer          OK  320 ms
09:00:03  Drive: create folder              RETRY
09:00:05  Drive: create folder              RETRY
09:00:10  Drive: create folder              OK
09:00:10  Human approval requested          WAITING
```

Initial engineering SLOs

These are **recommended targets**, not claims about the underlying OSS projects.

Service objective	Initial target
Control-plane API monthly availability	99.9%
Webhook durable-acceptance availability	99.95%

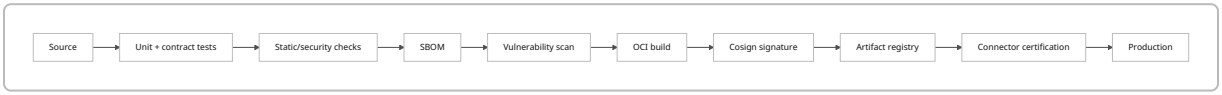
Service objective	Initial target
Internal gateway/control overhead p95	<250 ms
Webhook durable acknowledgement p95	<2 seconds
JetStream event dispatch p95	<2 seconds in healthy cluster
Normal CDC lag p95	<10 seconds where source permits
Trace coverage for production runs	>99.9%
Missing actor attribution	0
Secret values in application logs	0
Undetected cross-tenant result	0

External SaaS latency should be reported separately from our internal latency.

Connector supply-chain model

Connectors are executable software with access to customer systems.

Treat each connector release like a product release.



Use **Syft** to generate CycloneDX/SPDX SBOMs. Syft is Apache-2.0 and supports SBOM generation from container images, filesystems and many package ecosystems. [48]

Use **Grype or an equivalent scanner** to scan images, filesystems and SBOMs against known vulnerabilities. [49]

Use **Cosign** for OCI artifact signing. Cosign is Apache-2.0, supports keyless/KMS-backed signing, and its project documentation explicitly recommends signing immutable image digests rather than mutable tags. [50]

Example deployment reference:

```
BAD:
registry.companyos.io/connectors/twenty:latest

GOOD:
registry.companyos.io/connectors/twenty@sha256:b86d...
```

Admission requirements

Production connector artifacts should be rejected unless:

```
Manifest valid
AND
license permitted
AND
SBOM present
AND
artifact signed by approved publisher
AND
digest matches registration
AND
critical vulnerability policy passes
AND
integration tests pass
AND
capability declaration approved
```

Runtime sandbox

Untrusted/third-party workers should have:

- Non-root user.
- Read-only root filesystem.
- Temporary writable volume only when necessary.
- Memory limit.
- CPU limit.
- Process limit.
- No host filesystem.
- No Docker socket.
- No Kubernetes credentials.
- Network egress allowlist.
- No ambient tenant secrets.
- Short-lived execution token.
- Seccomp/AppArmor or equivalent runtime policy.

Activepieces import policy

Because Activepieces mixes MIT Community Edition with commercially licensed enterprise areas, importing code must have an automated path/license check. [32]

A piece is not production-approved just because its source is available.

Review:

```
source license
dependency licenses
current vendor API
authentication
token scopes
rate limits
error handling
pagination
PII behavior
tests
```

Upgrade strategy

Never auto-upgrade an integration engine immediately across the whole fleet.

Use:

```
upstream release
    ↓
compatibility environment
    ↓
our regression suite
    ↓
security review
    ↓
internal canary
    ↓
1–5% tenant canary
    ↓
controlled rollout
    ↓
old runtime drained
```

Camel: remain on a supported LTS for production connector workers unless a newer feature line is specifically needed. Camel maintains upgrade recipes to assist migrations between release families. [51]

APISIX: read each release's breaking-change section. APISIX 3.17 explicitly includes compatibility-impacting changes in authentication, TLS and configuration behavior. [52]

Temporal: server and SDK upgrade compatibility must be tested independently of workflow code. The current 1.31 release line has GA Worker Deployment Versioning facilities, which can help stage worker-code updates. [53]

Debezium: preserve offsets and schema history through upgrades. Connector upgrades must be tested against the database version and schema being captured. Debezium explicitly requires persistent state for connector offsets and, for several databases, schema history. [45]

SeaTunnel: review incompatible-change documentation before upgrades rather than assuming job compatibility. [54]

MCP: store protocol version per server connection. MCP now has a formal evolving specification, so the broker must support protocol negotiation/adaptation rather than assuming today's transport will remain unchanged forever. The July 2026 release itself represents substantial transport/auth changes from earlier versions. [55]

Playwright: browser engine versions move frequently; browser automations should therefore have automated regression runs against the runtime version being promoted. [9]

Developer APIs, SDKs, flow definitions, and implementation tasks

The developer experience should expose **our abstraction**, regardless of whether Camel, Temporal or Playwright runs underneath.

Public Integration Control Plane API

Recommended resource model:

Endpoint	Purpose
`GET /v1/connectors`	Connector catalog
`GET /v1/connectors/{id}`	Connector detail
`GET /v1/connectors/{id}/versions`	Versions
`POST /v1/connections`	Create connection
`POST /v1/connections/{id}/verify`	Test connection
`POST /v1/connections/{id}/oauth/start`	Begin OAuth
`DELETE /v1/connections/{id}`	Revoke/remove
`GET /v1/connections/{id}/capabilities`	Enabled operations
`POST /v1/capabilities/{id}:execute`	Execute capability
`POST /v1/workflows`	Create flow
`POST /v1/workflows/{id}/versions`	Publish revision
`POST /v1/workflows/{id}:activate`	Activate
`POST /v1/workflows/{id}:pause`	Pause

Endpoint	Purpose
`POST /v1/workflows/{id}/runs`	Start run
`GET /v1/runs/{id}`	Inspect run
`POST /v1/runs/{id}:cancel`	Cancel
`POST /v1/runs/{id}:retry`	Retry failure
`POST /v1/runs/{id}:replay`	Replay
`GET /v1/runs/{id}/steps`	Step trace
`POST /v1/mappings`	Create mapping
`POST /v1/mappings/{id}:test`	Test mapping
`POST /v1/mcp/connections`	Register MCP server
`POST /v1/mcp/connections/{id}:discover`	Discover tools/resources
`POST /v1/mcp/connections/{id}:approveChanges`	Approve tool changes
`GET /v1/events/subscriptions`	Event subscriptions
`POST /v1/webhook-endpoints`	Create incoming webhook

Capability execution API

Request:

```
POST /v1/capabilities/crm.company.create:execute
Idempotency-Key: 90dd...

{
  "connectionId": "conn_twenty_01",
  "input": {
    "name": "Example AB",
    "domain": "example.se"
  },
  "context": {
    "customerId": "cust_221",
    "projectId": "proj_71"
  }
}
```

Response:

```
{
  "runId": "run_842",
  "status": "succeeded",
  "output": {
    "id": "external_183",
    "name": "Example AB"
  },
  "external": {
    "system": "twenty",
    "requestId": "..."
  }
}
```

The caller should **not supply an authoritative tenant or actor ID in the body**. Those should come from the authenticated request context.

Connector SDK

I recommend a **TypeScript SDK as the primary custom-connector SDK** because it matches the likely Company OS application stack and makes it easier to reuse selected Activepieces logic.

A Java/Camel adapter exposes the same contract for Camel components.

Conceptually:

```
export interface ConnectorContext {
  readonly tenantId: string;
  readonly connectionId: string;
  readonly runId: string;
  readonly traceId: string;

  http: ScopedHttpClient;
  secrets: ExecutionSecretHandle;
  checkpoint: CheckpointStore;
  idempotency: IdempotencyService;
  events: EventEmitter;
  logger: RedactingLogger;
}

export interface Action<I, O> {
  definition: ActionDefinition;
  run(ctx: ConnectorContext, input: I): Promise<O>;
}

export interface Trigger<O> {
```

```
  definition: TriggerDefinition;
  start(
    ctx: TriggerContext,
    emit: (event: O) => Promise<void>
  ): Promise<void>;
}
```

The SDK's `http` object should enforce allowed hosts from the connector manifest.

Do not expose unrestricted:

```
fetch(anyUrl)
```

to third-party connectors by default.

SDK project types

```
companyos-connector-sdk
├── HTTP/OpenAPI connector
├── webhook trigger
├── polling trigger
├── file trigger
├── database capability
├── Camel adapter
├── MCP adapter
├── Activepieces compatibility adapter
├── browser automation adapter
└── test harness
```

Connector CLI

Engineers should have:

```
companyos connector init
companyos connector validate
companyos connector test
companyos connector test-auth
companyos connector test-webhook
companyos connector sbom
companyos connector build
companyos connector sign
companyos connector publish
```

Local emulator

A developer should not need a production Company OS cluster to write a connector.

```
companyos dev connector ./connectors/acme
```

should provide fake:

- Tenant context.
- Secret broker.
- Idempotency service.
- Event bus.
- Checkpoint storage.
- Approval service.

Integration test contract

Every production connector should pass standard tests:

```
connect
authenticate
read
write
pagination
rate limit
timeout
token expiry
token revocation
bad input
provider 4xx
provider 5xx
duplicate request
network disconnect
schema variation
large response
PII logging
```

Example flow definition

```
apiVersion: companyos.io/workflow/v1
kind: IntegrationWorkflow

metadata:
  id: new-customer-onboarding
```

```yaml
  version: 7

trigger:
  type: event
  event: crm.customer.created

steps:
  - id: lookup-erp-customer
    capability: erp.customer.search
    connection: "${connections.erp}"
    input:
      email: "${trigger.customer.email}"

  - id: create-if-missing
    if: "${steps.lookup-erp-customer.output.count == 0}"
    capability: erp.customer.create
    connection: "${connections.erp}"
    idempotencyKey: "${run.id}:erp-customer"
    input:
      name: "${trigger.customer.name}"
      email: "${trigger.customer.email}"

  - id: create-drive-folder
    capability: documents.folder.create
    connection: "${connections.drive}"
    idempotencyKey: "${run.id}:drive-folder"
    input:
      name: "${trigger.customer.name}"

  - id: request-approval
    type: approval
    when: "${trigger.customer.riskScore >= 8}"
    approverRole: customer-risk-manager

  - id: publish
    type: event
    event: customer.onboarding.started
```

Temporal can compile/execute the durable control logic while Camel/native workers perform the individual external capabilities.

Example MCP exposure

The same capability can be exposed to AI through MCP:

```
Native capability:
crm.company.search

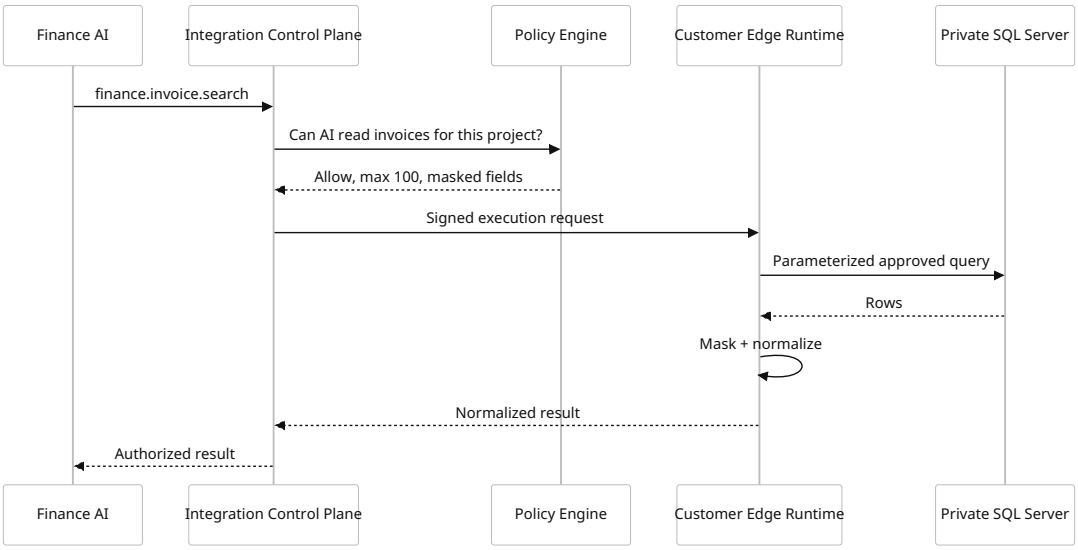
MCP tool presented to permitted AI:
companyos.crm.company.search
```

The model sees only the schema and capability allowed by policy.

It does not receive:

- CRM API key.
- Vendor admin token.
- Database password.
- Internal connector endpoint.

Example private database execution



ERPNext/Frappe adapter

Frappe currently generates REST endpoints for all DocTypes and supports RPC-style whitelisted methods, making an API-first ERPNext/Frappe connector practical. [56]

The connector should support:

```
doctype.search
doctype.read
doctype.create
```

```
doctype.update
method.call
webhook.subscribe/configure
```

but expose higher-level Company OS capabilities where stable semantics exist.

Twenty adapter

Twenty currently exposes both REST and GraphQL APIs generated from the workspace schema, including custom objects, plus webhooks and OAuth. [57]

This makes Twenty useful for testing:

- REST.
- GraphQL.
- Dynamic schemas.
- Rate limits.
- Custom objects.
- Webhooks.
- OAuth/API-key patterns.

Developer implementation workstreams

Workstream	Main engineering tasks
Control Plane	Data model, API, versions, capabilities, connections
Security	Policy engine, execution tokens, secret broker
Gateway	APISIX configuration/control integration
Connector SDK	TS SDK, CLI, emulator, test harness
Camel runtime	Worker contract, route templates, health
Temporal	Workflow compiler/runtime, approvals, cancellation
Events	NATS/JetStream, schemas, outbox/inbox
MCP	Client broker, server exposure, discovery/governance
Edge Runtime	Registration, outbound control channel, updater
CDC	Debezium orchestration and normalization
Bulk	SeaTunnel job adapter
Browser	Playwright sandbox/worker
Mapping	Schema registry, mapping language/editor
Observability	OTel, run traces, metrics, alerting

Workstream	Main engineering tasks
Supply chain	Build/sign/scan/certification
UI	Connections, catalog, workflows, mapping, runs, incidents

Proof of concept, acceptance criteria, risk matrix, effort, and implementation checklist

The POC should prove the architecture, not merely demonstrate three successful API calls.

Recommended POC topology

```
Company OS Integration Control Plane
├── PostgreSQL
├── OPA
├── APISIX
├── Temporal
├── NATS JetStream
├── Camel workers
├── MCP Broker
├── connector registry
├── local encrypted secret provider
└── OpenTelemetry

Test systems
├── ERPNext / Frappe
├── Twenty
├── Gmail / Google Drive
├── remote PostgreSQL
├── Debezium
├── mock MCP server
└── mock legacy web app
```

SeaTunnel can be a separate POC track rather than a hard dependency of the first architecture demonstration.

POC scenarios

Scenario	What it proves
ERPNext read/write through REST	API connector
ERPNext signed webhook	Webhook ingress
Twenty REST query	REST

Scenario	What it proves
Twenty GraphQL query	GraphQL
Twenty custom object discovery	Dynamic schema
Gmail/Drive operation	OAuth cloud integration
Remote PostgreSQL read	JDBC
Remote DB write with constrained user	DB governance
Debezium PostgreSQL CDC	CDC
NATS event fan-out	Event backbone
Temporal approval wait	Durable workflow
MCP external tool discovery	MCP client
Company capability exposed via MCP	MCP server
Legacy test site through Playwright	Browser fallback
Worker running in edge network	Private integration

Frappe supplies automatically generated DocType REST APIs and signed webhook support, so it is well suited for the API/webhook side of the POC. [58]

Twenty exposes both REST and GraphQL against workspace-specific schemas, so it provides a useful second API style and dynamic-schema test. [34]

Camel's current catalog includes Google Mail and Google Drive components, allowing Gmail/Drive to exercise OAuth-oriented cloud connectivity without making those services core architectural dependencies. [59]

Failure-injection test suite

The POC is not accepted until the following are exercised.

Failure	Expected behavior
Connector process killed before API request	Automatic retry
Worker killed after external write but before local ack	No duplicate business effect
Same webhook sent 100 times	One logical effect
Provider returns 429	Backoff, no DLQ yet
Provider returns permanent 400	Immediate permanent failure
OAuth token expires	Refresh where allowed

Failure	Expected behavior
OAuth connection revoked	Connection becomes unhealthy
Secret disabled	Execution blocked
NATS node restarted	Durable events continue
Temporal worker restarted	Workflow resumes
Edge runtime disconnected	Work waits/retries
Database inaccessible	Circuit opens after policy threshold
CDC service restarted	Resumes from durable offset
Duplicate CDC event	One downstream effect
Source schema adds field	Compatible handling
Source schema removes required field	Mapping failure/DLQ
MCP server adds destructive tool	Tool not automatically exposed
MCP server changes schema	Requires revalidation
Browser selector changes	Connector marked unhealthy
Cross-tenant ID injected	Access denied
AI attempts forbidden capability	Policy denies
Connector attempts unexpected network host	Egress denied

POC acceptance criteria

Tenancy

```
0 confirmed cross-tenant data leaks
0 ability to use another tenant's connection
0 ability to access another tenant's credential
```

Reliability

```
duplicate event tests produce one business effect
worker crash does not lose acknowledged business work
Temporal workflows resume after worker restart
CDC resumes from stored checkpoint
DLQ is inspectable and replayable
```

Security

```
no secret in logs
no secret in traces
no raw long-lived credential reaches an AI prompt
network egress policy blocks unauthorized destinations
unsigned connector image cannot be deployed
MCP tool changes do not bypass approval
```

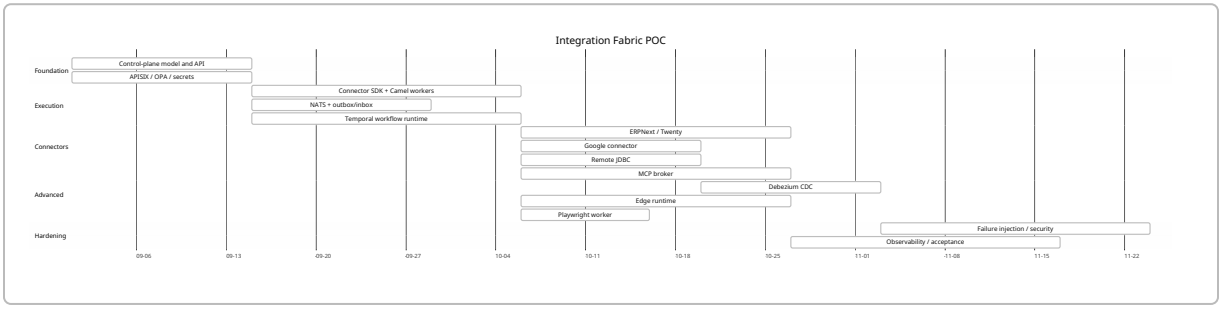
Observability

```
>99.9% of POC integration runs have a complete correlation path
every failed run identifies failing connector and step
external provider request IDs are retained when available
```

Developer experience

```
new basic REST connector can be created using SDK template
connector can be tested locally
manifest validation is automatic
SBOM generated automatically
signed artifact promoted automatically after tests
```

Suggested POC timeline



The dates are illustrative; the important point is the dependency order.

Estimated engineering effort

These are planning estimates, not promises.

Stage	Team	Calendar estimate	Person-effort
Architectural spike	2–3 senior engineers	2–3 weeks	5–8 person-weeks

Stage	Team	Calendar estimate	Person-effort
Credible POC	5–7 engineers	**10–14 weeks**	**50–80 person-weeks**
Hardened internal alpha	7–9 engineers	additional 3–4 months	100–150 person-weeks
Production v1 integration fabric	8–12 engineers + SRE/security	**6–9 months total after POC direction is validated**	**220–380 person-weeks**
Broad enterprise integration platform	10–15 engineers	12–18 months	**450–800+ person-weeks**

These estimates exclude building hundreds of polished vendor connectors.

Using Camel and selectively adapting Activepieces connectors should reduce connector-development effort substantially, but **every imported connector still needs recertification**.

Principal risks

Risk	Probability	Impact	Mitigation
Cross-tenant data leakage	Medium	**Critical**	Signed tenant context, worker isolation, policy tests
Credential leakage	Medium	**Critical**	Secret broker, redaction, no prompt credentials
Third-party connector compromise	Medium	**Critical**	SBOM, signing, sandbox, egress controls
Direct DB write damages customer system	Medium	**Critical**	Read-only default, approved capabilities, transactions
Duplicate side effects	High	High	Idempotency, inbox/outbox, reconciliation
Vendor API breaking changes	High	Medium–High	Versioned connectors, contract tests, canaries
MCP tool changes expand authority	High	High	Tool snapshots and reapproval
MCP malicious descriptions/tool metadata	Medium	High	Treat metadata as untrusted, local risk classification
CDC duplicate/change ordering issue	Medium	High	Idempotency, partition ordering, durable offsets
CDC schema drift	High	Medium–High	Schema registry, compatibility tests, DLQ

Risk	Probability	Impact	Mitigation
Browser UI changes	**Very high**	Medium–High	Last-resort status, health tests, evidence
Integration infrastructure becomes too complex	Medium	High	Do not deploy specialist engines until required
Temporal lock-in in workflow definitions	Medium	Medium	Keep Company OS workflow DSL independent
Camel route logic becomes business logic	Medium	High	Camel only executes connector operations
Activepieces license boundary accidentally crossed	Medium	High	Automated path/license scanning
Customer firewall prevents cloud worker access	High	High	Edge Runtime
Event “exactly once” assumption is wrong	High	High	Design for at-least-once and idempotency

Recommended production rollout order

The architecture should be implemented incrementally:

```
FIRST
Integration Control Plane
APISIX
OPA
Connector contract
Secrets abstraction
Camel
REST/OpenAPI
Webhooks
JDBC
NATS JetStream
ERPNext + Twenty

        ↓

SECOND
Temporal
MCP Broker
Google/Microsoft cloud connectors
Email
SFTP/SMB
Edge Runtime
```

```
Mapping framework

        ↓

THIRD
Debezium CDC
Advanced SOAP/gRPC
Connector marketplace/import pipeline
Kaoto-derived visual tools

        ↓

WHEN REQUIRED
SeaTunnel
High-volume streaming infrastructure
Playwright legacy automation
Dedicated tenant execution clusters
```

This avoids repeating the problem seen in Open Integration Hub: deploying a large collection of infrastructure services before actual customer workloads require all of them.

Implementation checklist

Area	Required before production
Architecture	☐ Control plane independent from execution engines
	☐ Stable Company OS Capability model
	☐ Connectors and connections separated
	☐ Flow model independent of Temporal
Tenancy	☐ Server-derived tenant context
	☐ No caller-controlled authoritative tenant IDs
	☐ Cross-tenant automated attack tests
	☐ Per-tenant connection ownership
AI governance	☐ Human/AI/service principal differentiation
	☐ AI capability limits
	☐ Delegation chain
	☐ Approval gates for high-risk actions
Secrets	☐ KMS/envelope encryption
	☐ Secrets referenced, never stored in manifests

Area	Required before production
	☐ OAuth rotation/revocation
	☐ No credentials in prompts/logs/events
Connectors	☐ Versioned connector manifests
	☐ JSON Schema inputs/outputs
	☐ Side-effect classification
	☐ Idempotency declaration
	☐ Rate-limit declaration
	☐ Network egress declaration
Runtime	☐ Camel execution provider
	☐ Temporal durable provider
	☐ JetStream durable events
	☐ Execution cancellation/timeouts
	☐ Circuit breakers
Reliability	☐ Transactional outbox
	☐ Consumer inbox/dedupe
	☐ Retry taxonomy
	☐ DLQ
	☐ Manual replay
	☐ Ambiguous-write reconciliation
MCP	☐ Current protocol support
	☐ OAuth/resource validation
	☐ Server/tool snapshots
	☐ Tool-change review
	☐ Risk classification independent of server metadata
	☐ No token passthrough
	☐ stdio servers sandboxed
Database	☐ Read-only default
	☐ Schema/table allowlists

Area	Required before production
	☐ Parameterized capabilities
	☐ Query timeout
	☐ Dedicated database identity
CDC	☐ Durable offsets
	☐ Durable schema history
	☐ Sensitive field filtering
	☐ Duplicate-event testing
	☐ Schema-drift handling
Edge Runtime	☐ Outbound-only connectivity option
	☐ mTLS/device identity
	☐ Remote revoke
	☐ Signed updates
	☐ Local execution isolation
Browser	☐ Isolated Playwright workers
	☐ Domain restrictions
	☐ Separate browser contexts
	☐ Screenshot/trace evidence
	☐ Automated health regression
Mapping	☐ Immutable mapping versions
	☐ Fixtures and golden tests
	☐ Schema compatibility
	☐ Currency/date/timezone testing
Observability	☐ OpenTelemetry
	☐ Correlation IDs
	☐ Run/step trace UI
	☐ Vendor IDs captured
	☐ Queue/CDC/browser metrics
Supply chain	☐ SBOM

Area	Required before production
	☐ Vulnerability scan
	☐ Licence scan
	☐ OCI digest pinning
	☐ Cosign signature verification
	☐ Egress policy
Upgrade	☐ Connector canary mechanism
	☐ Runtime compatibility suite
	☐ Rollback
	☐ Temporal worker-version strategy
	☐ MCP protocol-version compatibility

The final architectural principle should be kept very strict:

```
                COMPANY OS

     knows companies, humans, AI employees,
     scopes, roles, authority and business meaning

                    │
                    ▼

          INTEGRATION CONTROL PLANE

    knows capabilities, connections, policies,
    mappings, flows, runs and external identities

                    │
                    ▼

             EXECUTION PROVIDERS

Camel │ Temporal │ NATS │ MCP │ Debezium │ SeaTunnel │ Playwright

                    │
                    ▼

               ANY SYSTEM
```

```
API | DB | MCP | SOAP | GraphQL | gRPC | Mail | SFTP | SMB
Cloud | Webhook | Queue | CDC | Files | Browser
```

Camel, Temporal, NATS, Debezium, APISIX and MCP are infrastructure. The strategic product is the governed semantic layer above them.

That is what allows the Company OS to remain stable when one customer uses Twenty and ERPNext, another uses Salesforce and SAP, another uses Notion and Microsoft 365, and another still runs a fifteen-year-old SQL database and exchanges invoices through SFTP.

The Company OS should see **capabilities and company authority**.

The integration fabric should absorb the technical differences.

---

[1] [10] Components - Apache Camel
https://camel.apache.org/components/4.18.x/index.html?utm_source=chatgpt.com

[2] Downloads - Apache Camel
https://camel.apache.org/download/?utm_source=chatgpt.com

[3] [52] Release Apache APISIX 3.17.0
https://apisix.apache.org/blog/2026/06/15/release-apache-apisix-3.17.0/?utm_source=chatgpt.com

[4] GitHub - temporalio/temporal: Temporal service · GitHub
https://github.com/temporalio/temporal?utm_source=chatgpt.com

[5] [28] JetStream | NATS Docs
https://docs.nats.io/nats-concepts/jetstream?utm_source=chatgpt.com

[6] [22] [35] Specification and documentation for the Model Context ...
https://github.com/modelcontextprotocol/modelcontextprotocol?utm_source=chatgpt.com

[7] [17] Debezium 3.6.1.Final Released
https://debezium.io/blog/2026/08/04/debezium-3-6-1-final-released/?utm_source=chatgpt.com

[8] [19] [30] Releases · apache/seatunnel
https://github.com/apache/seatunnel/releases?utm_source=chatgpt.com

[9] [46] microsoft/playwright: Playwright is a framework ...
https://github.com/microsoft/playwright?utm_source=chatgpt.com

[11] GraphQL - Apache Camel
https://camel.apache.org/components/4.18.x/graphql-component.html?utm_source=chatgpt.com

[12] REST OpenApi - Apache Camel
https://camel.apache.org/components/4.18.x/rest-openapi-component.html?utm_source=chatgpt.com

[13] JDBC - Apache Camel
https://camel.apache.org/components/4.18.x/jdbc-component.html?utm_source=chatgpt.com

[14] oas-validator | Apache APISIX
https://apisix.apache.org/docs/apisix/plugins/oas-validator/?utm_source=chatgpt.com

15 Webhooks - Frappe Documentation
https://docs.frappe.io/framework/user/en/guides/integration/webhooks?utm_source=chatgpt.com

16 SFTP - Apache Camel
https://camel.apache.org/components/4.18.x/sftp-component.html?utm_source=chatgpt.com

18 Releases · nats-io/nats-server · GitHub
https://github.com/nats-io/nats-server/releases?utm_source=chatgpt.com

20 Kaoto 2.11 release - The Integration Designer for Apache Camel
https://kaoto.io/blog/2026/kaoto-2.11-release/?utm_source=chatgpt.com

21 33 Releases · activepieces/activepieces · GitHub
https://github.com/activepieces/activepieces/releases

23 open-policy-agent/opa
https://github.com/open-policy-agent/OPA?utm_source=chatgpt.com

24 42 Resilience4j EIP - Apache Camel
https://camel.apache.org/components/4.18.x/eips/resilience4j-eip.html?utm_source=chatgpt.com

25 Apache Camel Security Advisory - CVE-2026-46590 - Apache Camel
https://camel.apache.org/security/CVE-2026-46590.html?utm_source=chatgpt.com

26 openid-connect - Apache APISIX
https://apisix.apache.org/docs/apisix/plugins/openid-connect/?utm_source=chatgpt.com

27 Multi-tenancy - Temporal feature
https://docs.temporal.io/evaluate/development-production-features/multi-tenancy?utm_source=chatgpt.com

29 Source Connectors :: Debezium Documentation
https://debezium.io/documentation/reference/stable/connectors/index.html?utm_source=chatgpt.com

31 GitHub - KaotoIO/kaoto: The UI of the Kaoto project · GitHub
https://github.com/KaotoIO/kaoto

32 GitHub - activepieces/activepieces: AI Agents & MCPs & AI Workflow Automation • (~400 MCP servers for AI agents) • AI Automation / AI Agent with MCPs • AI Workflows & AI Agents • MCPs for AI Agents · GitHub
https://github.com/activepieces/activepieces?utm_source=chatgpt.com

34 57 APIs - Twenty Documentation
https://docs.twenty.com/developers/extend/api?utm_source=chatgpt.com

36 Tools
https://modelcontextprotocol.io/specification/2026-07-28/server/tools?utm_source=chatgpt.com

37 Authorization
https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization?utm_source=chatgpt.com

38 Specification
https://modelcontextprotocol.io/specification/2026-07-28?utm_source=chatgpt.com

39 Elicitation
https://modelcontextprotocol.io/specification/2026-07-28/client/elicitation?utm_source=chatgpt.com

40 Overview
https://modelcontextprotocol.io/specification/2026-07-28/basic?utm_source=chatgpt.com

41 Debezium connector for SQL Server
https://debezium.io/documentation/reference/stable/connectors/sqlserver.html?utm_source=chatgpt.com

43 Build a Superfast CDC with Debezium, Quarkus and ...
https://debezium.io/blog/2025/03/12/superfast-debezium/?utm_source=chatgpt.com

44 Debezium Engine
https://debezium.io/documentation/reference/stable/development/engine.html?utm_source=chatgpt.com

45 Storing state of a Debezium connector
https://debezium.io/documentation/reference/stable/configuration/storage.html?utm_source=chatgpt.com

47 Context propagation
https://opentelemetry.io/docs/concepts/context-propagation/?utm_source=chatgpt.com

48 GitHub - anchore/syft: CLI tool and library for generating a Software Bill of Materials from container images and filesystems · GitHub
https://github.com/anchore/syft?utm_source=chatgpt.com

49 anchore/grype: A vulnerability scanner for container ...
https://github.com/anchore/grype?utm_source=chatgpt.com

50 GitHub - sigstore/cosign: Code signing and transparency for containers and binaries · GitHub
https://github.com/sigstore/cosign?utm_source=chatgpt.com

51 apache/camel-upgrade-recipes
https://github.com/apache/camel-upgrade-recipes?utm_source=chatgpt.com

53 Releases · temporalio/temporal · GitHub
https://github.com/temporalio/temporal/releases?utm_source=chatgpt.com

54 Incompatible Changes - Apache SeaTunnel
https://seatunnel.apache.org/docs/introduction/concepts/incompatible-changes/?utm_source=chatgpt.com

55 The 2026-07-28 Specification
https://blog.modelcontextprotocol.io/posts/2026-07-28/?utm_source=chatgpt.com

56 58 REST API - Frappe Documentation
https://docs.frappe.io/framework/user/en/api/rest?utm_source=chatgpt.com

59 Tika - Apache Camel
https://camel.apache.org/components/4.18.x/tika-component.html?utm_source=chatgpt.com